Atelier 7 : Création d'une Image Docker avec Nginx

1. Introduction

Dans le cadre de ce TP, nous avons étudié l'utilisation de **Docker**, une plateforme de conteneurisation permettant de déployer des applications de manière rapide, portable et isolée.
L'objectif principal de cet atelier est de créer une image Docker personnalisée contenant un serveur web **Nginx**, puis d'exécuter un conteneur à partir de cette image.

Ce TP a été réalisé à l'aide de **Visual Studio Code** et d'un **Dockerfile**, ce qui correspond aux bonnes pratiques professionnelles.

2. Objectifs du TP

Les objectifs de cet atelier sont les suivants :

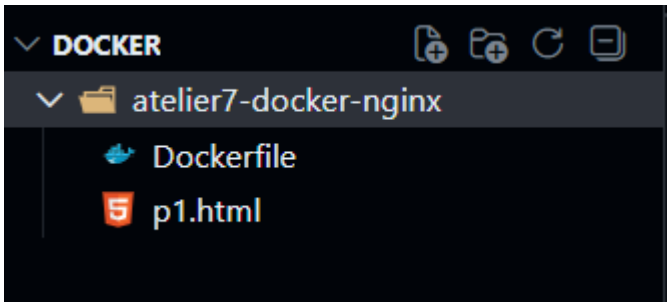
- Installer et utiliser Docker
- Créer une image Docker personnalisée
- Utiliser un Dockerfile pour automatiser la création de l'image
- Déployer un serveur web Nginx dans un conteneur
- Accéder à une page HTML servie par Nginx via un navigateur

3. Environnement de travail

- Système d'exploitation : Windows
- Outil de conteneurisation : Docker Desktop
- Éditeur de code : Visual Studio Code
- Serveur web : Nginx

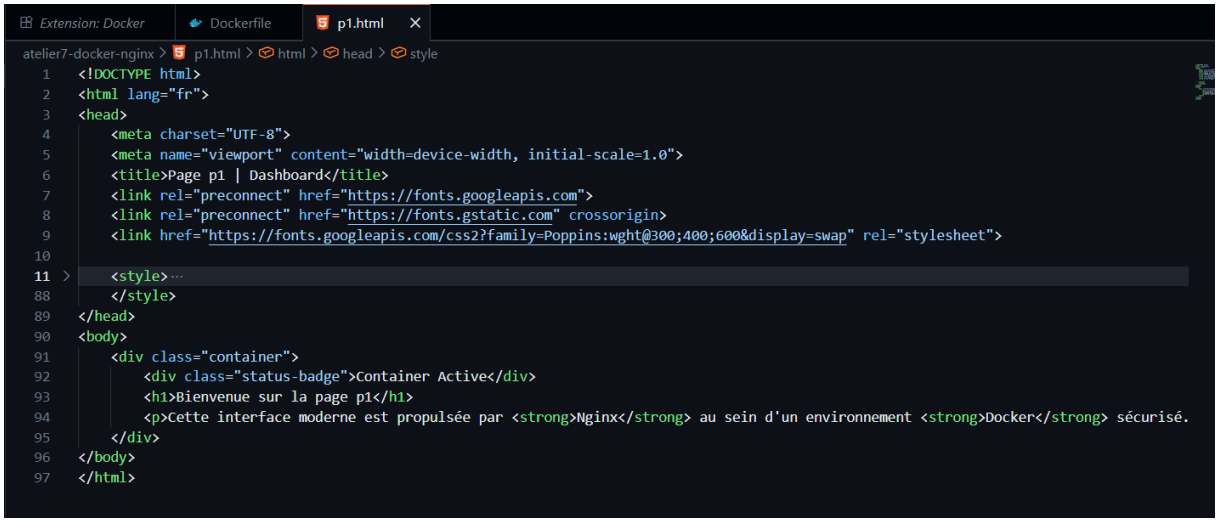
4. Structure du projet

Le projet est organisé de la manière suivante :



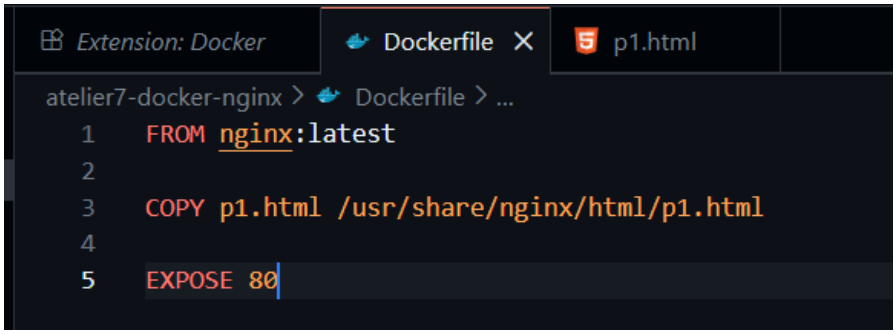
5. Création de la page HTML

Le fichier p1.html contient une page web simple affichée par le serveur Nginx :



6. Création du Dockerfile

Le Dockerfile permet de créer automatiquement l'image Docker :



Explication :

- FROM nginx:latest : utilise l'image officielle Nginx
- COPY : copie la page HTML dans le dossier public de Nginx
- EXPOSE 80 : expose le port 80 du conteneur

7. Construction de l'image Docker

La commande suivante permet de construire l'image Docker :

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS          powershell - atelier7-docker-nginx

PS C:\Users\Dell PC\Documents\docker> cd "atelier7-docker-nginx"
PS C:\Users\Dell PC\Documents\docker\atelier7-docker-nginx> docker build -t msbenyaishe/ngin
x-image:v1 .
[+] Building 13.8s (8/8) FINISHED                                   docker:desktop-linux
 => [internal] load build definition from Dockerfile                                0.0s
 => => transferring dockerfile: 113B                                                0.0s
 => [internal] load metadata for docker.io/library/nginx:latest                     3.9s
 => [auth] library/nginx:pull token for registry-1.docker.io                        0.0s
 => [internal] load .dockerignore                                                   0.0s
 => => transferring context: 2B                                                     0.0s
 => [internal] load build context                                                   0.0s
 => => transferring context: 3.27kB                                                 0.0s
 => [1/2] FROM docker.io/library/nginx:latest@sha256:0236ee02dcbce00b9bd83e0f5fbc51069e7  9.2s
 => => resolve docker.io/library/nginx:latest@sha256:0236ee02dcbce00b9bd83e0f5fbc51069e7  0.0s
 => => sha256:df0b66c867e491634d63f065ef5ea556cba20a97761039f9e9d5af7144 1.40kB / 1.40kB  0.4s
 => => sha256:35ff83c394d65bf2c49f844ba18fe2e8121fe07e1f3f92325c05246319b98f 405B / 405B  0.5s
 => => sha256:eedda9fd87862df4b2eccd3da9f2e38122124dec2e4217b530bd48d809b384 954B / 954B  0.6s
 => => sha256:17d0911eaf62f5c4c34e54943d30dbb0d86b8ad614c80ef5ee0249ff54 1.21kB / 1.21kB  0.7s
 => => sha256:b47f187216b68c3f90b7fb9a947a76d4bd2981a8f469cb6092e7da5d 33.14MB / 33.14MB  5.1s
enyaishe                    Ln 87, Col 10   Spaces: 4   UTF-8   CRLF   {} HTML   Port : 5500
```

Cette commande génère une image personnalisée nommée nginx-image version v1.

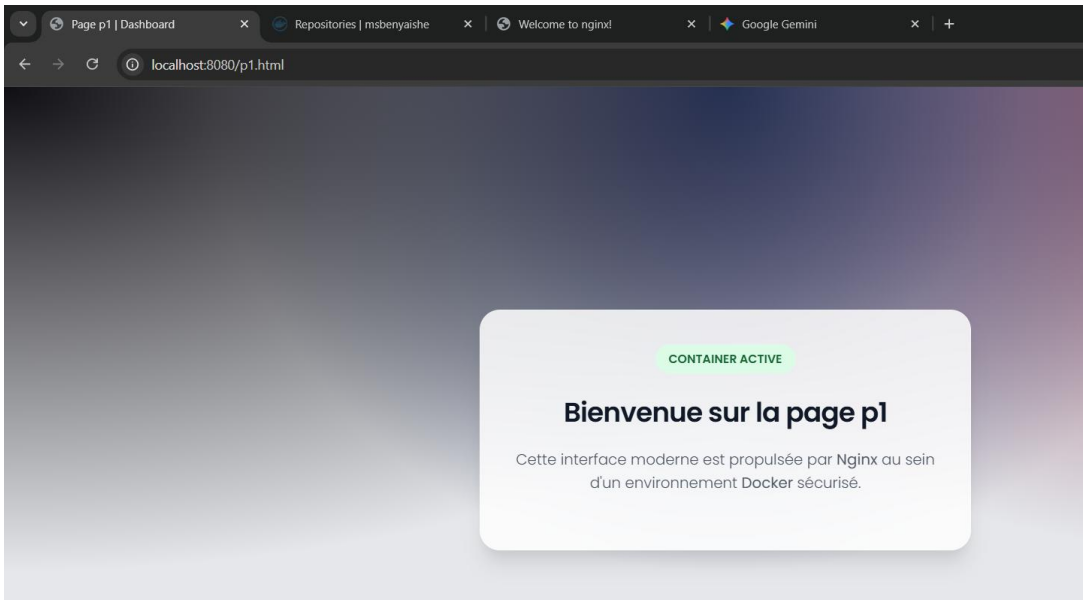
8. Exécution du conteneur

```
PROBLEMS   OUTPUT   TERMINAL   ...          powershell - atelier7-docker-nginx

PS C:\Users\Dell PC\Documents\docker\atelier7-docker-nginx> docker run -d -p 8080:80 --name ate
lier7-nginx msbenyaishe/nginx-image:v1
712d17b078ec931de04099f21c9b7c45c23086b6cb6120a6b7ac3695ec3e72fc
```

9. Test du serveur web

Après l'exécution du conteneur, la page web est accessible via le navigateur à l'adresse :



10. Résultats obtenus

À la fin de ce TP, nous avons réussi à :

- Créer une image Docker personnalisée
- Déployer un serveur Nginx dans un conteneur
- Servir une page HTML via Docker
- Comprendre le fonctionnement de Dockerfile et du mapping des ports

11. Conclusion

Ce TP nous a permis de découvrir les bases de Docker et de la conteneurisation.
L'utilisation d'un Dockerfile rend la création d'images reproductible et conforme aux bonnes pratiques professionnelles.
Docker facilite grandement le déploiement rapide d'applications web comme Nginx sur différents environnements.